Part 2 – Supervised Machine Learning: Build, Train and Evaluate

Project Overview

This project is about building machine learning models using a cleaned used car dataset. We developed two models:

1. Linear Regression to predict the selling price of a used car.

2. Logistic Regression to classify whether a cars selling price is above or below the price.

The project includes data preprocessing, feature encoding, feature scaling regression analysis, classification, regularization, ROC analysis, decision threshold evaluation and bootstrap confidence interval estimation.

Dataset

We used the cleaned dataset (`cleaned_data.csv`) from Part 1.

Number of records: 4009

Target variable for regression: price

Target variable for classification:

```python y_clf = (price > median(price)).astype(int) ```

We labeled cars priced above the median as 1 and cars priced at or below the median as 0.

Feature Engineering

The feature matrix X contains every column except price.

The regression target is price.

The binary classification target is price > median(price).

Encoding Categorical Variables

The dataset has variables:

- brand
- model
- fuel_type
- engine
- transmission
- accident
- clean_title
- exterior color
- color
```

We applied ‘One-Hot Encoding’ using `pd.get_dummies(drop_first=True)`.

Why One-Hot Encoding?

Label encoding can imply a relationship, which's incorrect. One-hot encoding avoids this by creating features for each category.

Train-Test Split

We divided the dataset into:

- 80% Training Data
- 20% Testing Data

using `train_test_split(test_size=0.2, random_state=42)`.

Feature Scaling

We performed feature scaling using StandardScaler.

We fitted the scaler on the training data before transforming both training and testing sets.

Why?

Fitting the scaler on the dataset would introduce **data leakage**. Using training statistics ensures an evaluation.

Linear Regression

We trained a Linear Regression model to predict used car prices.

Evaluation metrics: Mean Squared Error (MSE) and R^2 Score.

We extracted model coefficients. Matched them with feature names.

We identified the three features with the absolute coefficients.

Interpretation

- A positive coefficient means increasing the feature value increases the predicted selling price.
- A negative coefficient means increasing the feature value decreases the predicted selling price.

The larger the coefficient magnitude, the influence that feature has on the prediction.

Ridge Regression

We trained a Ridge Regression model with `alpha = 1.0`.

We compared metrics with Linear Regression: Mean Squared Error and R^2 Score.

Why Ridge Regression?

Linear Regression can produce coefficients with correlated features. Ridge Regression introduces a L2 penalty shrinking coefficient values and reducing model variance.

Classification using Logistic Regression

We trained a Logistic Regression model to classify cars as:

- Above Median Price
- Below Median Price

Before training we examined class balance. Since the dataset was imbalanced we used `class_weight="balanced"`.

Evaluation Metrics

We computed classification metrics:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1 Score

using `classification_report()`.

Precision and Recall

Precision measures how many predicted positive cars were actually positive.

Recall measures how many actual positive cars were correctly identified.

For this project Recall is more important.

ROC and AUC

We generated predicted probabilities using `predict_proba()`.

The ROC curve shows the relationship between Positive Rate and False Positive Rate.

The Area Under the Curve (AUC) measures the models ability to distinguish between classes.

Decision Threshold Analysis

We evaluated five thresholds: 0.30, 0.40 0.50 0.60 0.70.

For each threshold we calculated Precision, Recall, F1 Score.

We identified the threshold producing the best F1 Score.

Logistic Regression Regularization

We trained a Logistic Regression model with `C = 0.01`.

We compared metrics with the baseline model (C = 1.0): **Precision** **Recall**. **AUC**.

Meaning of C

C controls the strength of regularization. A larger C means regularization and a smaller C means stronger regularization.

Bootstrap Confidence Interval

We generated 500 bootstrap samples. Computed the AUC difference for each sample.

We calculated the AUC difference. Estimated the 95% confidence interval.

Libraries Used

- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn